

Table of Contents

1. INTRODUCTION & PROJECT OVERVIEW.....	3
1.1 Purpose of This Document.....	3
1.2 Project Overview.....	3
1.3 Background.....	3
1.4 Problem Statement.....	4
1.5 Research Objectives.....	4
1.6 Scope & Boundaries.....	4
2. OVERALL SYSTEM DESCRIPTION.....	5
2.1 System Perspective.....	5
2.2 User Classes & Characteristics.....	5
2.3 Operating Environment.....	6
2.4 Design & Implementation Constraints.....	6
2.5 Assumptions & Dependencies.....	6
3. FUNCTIONAL REQUIREMENTS (FR).....	7
3.1 Authentication Module.....	7
3.2 Youth Profile Module.....	7
3.3 Job Provider Module.....	8
3.4 Job Discovery & Matching Module.....	8
3.5 Application Module.....	8
3.6 Ratings & Reviews Module.....	9
3.7 Notifications Module.....	9
3.8 Admin Module.....	9
4. NON-FUNCTIONAL REQUIREMENTS (NFR).....	11
5. SYSTEM ARCHITECTURE.....	12
5.1 Architecture Overview.....	12
5.2 Layer Specifications.....	12
Layer 1 — Presentation (Flutter App).....	12
Layer 2 — Application Logic (Cloud Functions).....	12
Layer 3 — Data (Firebase Services).....	13
Layer 4 — External / Infrastructure.....	13
5.3 Data Flow — Job Posting to Youth Notification.....	13
5.4 Sequence — Youth Applies for a Job.....	15
6. DATA MODELS & FIRESTORE SCHEMA.....	16
6.1 Collection: users.....	16
6.2 Collection: jobs.....	17
6.3 Collection: youth_profiles.....	18
6.4 Collection: applications.....	18
6.5 Collection: ratings.....	18
6.6 Collection: notifications.....	19
6.7 Collection: skills_taxonomy.....	19
7. BUSINESS LOGIC & ALGORITHMS.....	21
7.1 Matching Engine (Cloud Function).....	21
Algorithm — Step by Step.....	21
Cloud Function Code Scaffold.....	21
7.2 Rating Average Update (Cloud Function).....	22
7.3 Application Status Notification (Cloud Function).....	22
7.4 Haversine Distance Formula.....	22
8. UI/UX SPECIFICATION & SCREEN INVENTORY.....	23
8.1 Visual Design Tokens.....	23
8.2 Screen Inventory.....	23
8.3 Key Screen Specifications.....	25
S07 — Home / Job Feed.....	25
S08 — Job Detail.....	25
9. STITCH MCP + ANTIGRAVITY INTEGRATION GUIDE.....	27
9.1 What is Stitch MCP + Antigravity?.....	27

9.2 Prerequisites.....	27
9.3 Screen Generation Workflow.....	27
Step 1 — Upload Design System.....	27
Step 2 — Generate Each Screen.....	27
Step 3 — Save Generated Widget Code.....	28
Step 4 — Wire Business Logic.....	28
10. FIREBASE CONFIGURATION & SETUP.....	29
10.1 Firebase Project Setup.....	29
10.2 Services to Enable.....	29
10.3 Required Flutter Packages.....	29
11. FLUTTER PROJECT STRUCTURE.....	31
11.1 Folder Structure.....	31
12. API & INTEGRATION SPECIFICATIONS.....	32
12.1 Firebase Firestore Queries.....	32
12.2 Google Maps Platform.....	32
12.3 FCM Notification Payload.....	32
13. SECURITY & ACCESS CONTROL.....	33
13.1 Firestore Security Rules.....	33
13.2 Firebase Auth Custom Claims.....	34
13.3 Firebase Storage Rules.....	34
14. TESTING STRATEGY & TEST CASES.....	35
14.1 Testing Levels.....	35
14.2 Critical Test Cases.....	35
14.3 Usability Testing Protocol.....	36
15. DEPLOYMENT & CI/CD.....	37
15.1 Environments.....	37
15.2 GitHub Actions CI/CD Pipeline.....	37
15.3 Pre-Submission Checklist.....	37
16. DEVELOPMENT CONSTRAINTS & ASSUMPTIONS.....	38
16.1 Technical Constraints.....	38
16.2 Academic Constraints.....	38
16.3 Assumptions.....	38
17. GLOSSARY.....	39
18. REFERENCES.....	40

1. INTRODUCTION & PROJECT OVERVIEW

1.1 Purpose of This Document

This Software Requirements Specification (SRS) is the authoritative reference document for the design, development, testing, and deployment of Fursafy — a mobile digital platform that connects youth to short-term informal job opportunities in Tanzania. This document is written to be consumed directly by an AI coding agent (such as Claude Code or any autonomous development agent) and must be followed precisely.

Every section of this SRS is structured to give an AI agent complete, unambiguous instructions. No assumptions should be made that are not stated here. When a specification is incomplete or ambiguous, the agent must flag it for human review before proceeding.

AI AGENT INSTRUCTION: Read this entire SRS before writing a single line of code. Build a mental map of: (1) all screens, (2) all Firestore collections, (3) all Cloud Functions, and (4) the Stitch MCP + Antigravity UI pipeline. Only then begin scaffolding.

1.2 Project Overview

App Name	Fursafy
Tagline	Fursa kwa Vijana — Opportunities for Youth
Platform	Android (primary), iOS (secondary) — Flutter cross-platform
Backend	Firebase (Firestore, Auth, Cloud Functions, FCM, Storage, Analytics)
UI Generation	Antigravity (via Stitch MCP connector)
Target Users	Youth (18–35) seeking short-term work in Tanzania; Job Providers (individuals/SMEs); Admin
Geography	Urban Tanzania — Arusha, Dar es Salaam, Moshi (Phase 1)
Language	Swahili + English bilingual UI
Offline Support	Read-only offline mode (cached listings); FCM for connectivity restoration
Monetisation	Free for Phase 1 prototype; Premium tier planned post-launch

1.3 Background

Youth unemployment remains a persistent socio-economic challenge in Tanzania and across East Africa. Each year, approximately 850,000 young people enter the Tanzanian labour market, while only 40,000–50,000 formal employment opportunities are created (ILO, 2022). A large proportion of youth rely on the informal economy, engaging in short-term or one-time jobs such as technical repairs, cleaning services, tutoring, construction assistance, and other skill-based tasks.

Despite the availability of such informal job opportunities, access remains unstructured, relying on personal networks, word of mouth, or informal social media groups. Existing platforms (DayWaka,

Mchongoo) serve the general population but do not provide skill-based matching, structured youth profiles, or trust-building mechanisms tailored to the informal sector. Fursafy addresses this gap.

1.4 Problem Statement

There is no dedicated digital platform in Tanzania that: (1) enables skill-based matching between youth and short-term job providers; (2) provides location-aware job discovery for informal, physical tasks; (3) builds trust through verified profiles and a ratings system; or (4) delivers real-time job notifications to matched youth. Fursafy is designed to solve all four problems simultaneously.

1.5 Research Objectives

1. To assess the challenges faced by youth in accessing short-term informal job opportunities.
2. To analyse the needs of job providers in sourcing skilled workers for one-time or short-term tasks.
3. To design a digital platform that enables skill-based job matching between youth and job providers.
4. To implement a prototype system that allows job posting and worker discovery.
5. To evaluate the usability and effectiveness of the developed platform.

1.6 Scope & Boundaries

IN SCOPE for this SRS and the AI agent build:

- Youth and Job Provider mobile app (Flutter — Android & iOS)
- Firebase backend: Auth, Firestore, Cloud Functions, FCM, Storage
- Skill-based + location-based job matching (Cloud Function)
- Real-time push notifications (FCM)
- Ratings and reviews system (bilateral)
- Admin dashboard (Flutter web or separate admin app)
- UI generation via Antigravity connected through Stitch MCP

OUT OF SCOPE (do not implement):

- In-app payment processing or mobile money integration
- Full-time or formal employment listings
- Video calling or real-time chat (Phase 2)
- ML-based recommendation beyond rule-based matching (Phase 2)

2. OVERALL SYSTEM DESCRIPTION

2.1 System Perspective

Fursafy is a standalone mobile application backed by Firebase BaaS (Backend-as-a-Service). It operates in a client–server architecture: the Flutter app is the client, Firebase services are the server. A Firebase Cloud Function acts as the Matching Engine, running server-side logic on every new job post.

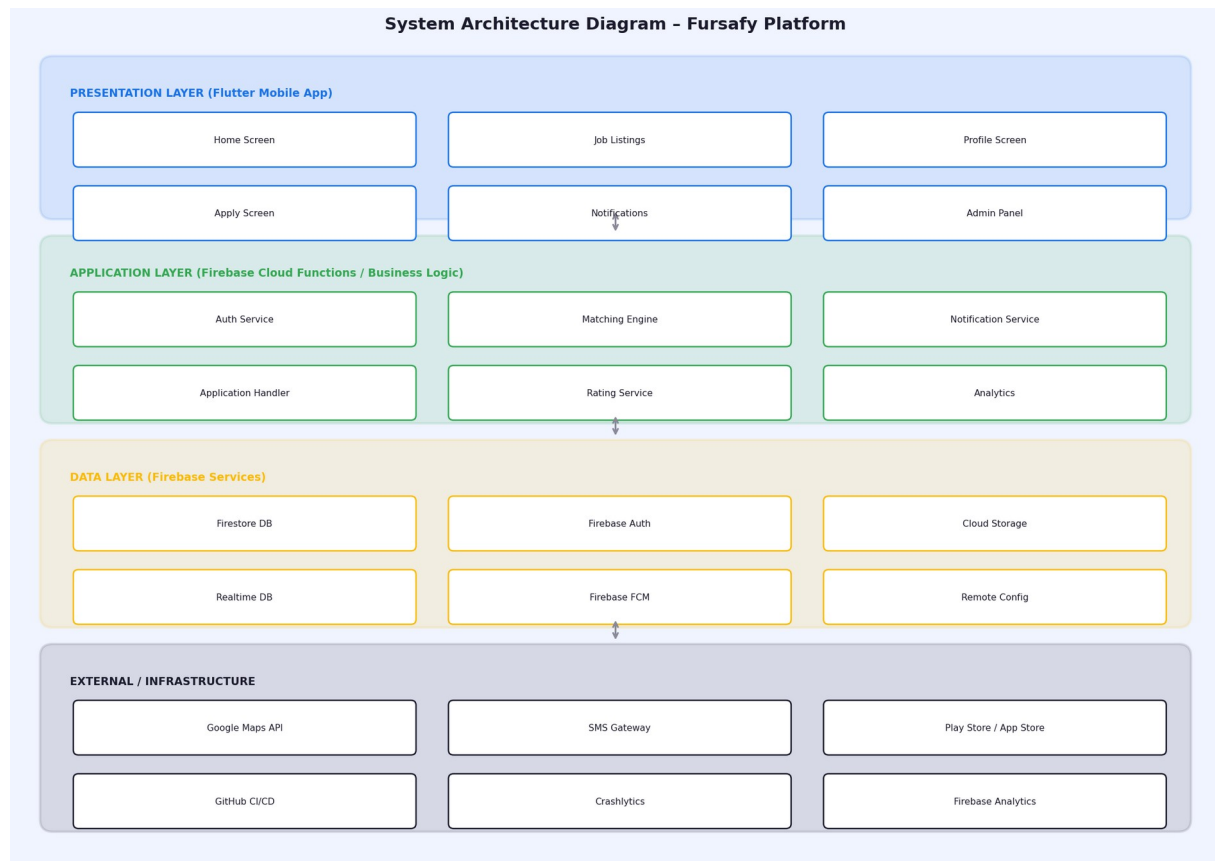


Figure 2.1 – Fursafy Four-Layer System Architecture

2.2 User Classes & Characteristics

User Class	Description	Technical Level	Primary Actions
Youth (Job Seeker)	Aged 18–35, urban Tanzania, smartphone user, may have low data bandwidth	Low to Medium	Register, build skill profile, browse/apply for jobs, receive matches, rate providers
Job Provider	Individual or SME needing short-term physical/service tasks done	Low	Register, post jobs, view applicants, select worker, rate youth
Admin	ATC or Platform operator; manages system integrity	High	Manage users, moderate listings, view analytics, export reports
System	Automated backend actor: Matching	N/A —	Trigger matching on new job,

User Class	Description	Technical Level	Primary Actions
(Firebase)	Engine, Notification Service	automated	send FCM push, update denormalised data

2.3 Operating Environment

- Client: Android 6.0+ (API 23+), iOS 13+ — Flutter 3.x
- Backend: Firebase (Google Cloud infrastructure, multi-region)
- Network: 3G/4G mobile data, WiFi; offline read-only mode for cached data
- IDE: Android Studio / VS Code / Antigravity with Flutter & Dart plugins
- Version control: Git + GitHub
- UI generation: Antigravity platform connected via Stitch MCP

2.4 Design & Implementation Constraints

- All UI components must be generated by Antigravity via Stitch MCP from the HiFi mockup specifications in Section 8.
- All Firebase Security Rules must be enforced — no client-side data access without rule validation.
- The app must be bilingual (Swahili + English). All user-facing strings must use Flutter's localisation framework (flutter_localizations).
- Firestore queries must stay within free Spark tier limits during academic prototype phase.
- No third-party payment SDK may be added in this version.

2.5 Assumptions & Dependencies

- Students have access to a Firebase project with Blaze plan enabled (required for Cloud Functions outbound network calls).
- Stitch MCP connector is available and authenticated against the Antigravity account.
- Google Maps API key is provisioned and restricted to the app's package name.
- Users have Android 6.0+ devices with internet access.
- Flutter SDK 3.19+ and Dart 3.x are installed in the development environment.

3. FUNCTIONAL REQUIREMENTS (FR)

AI AGENT: Every FR below must be implemented. FRs are grouped by module. Each FR has an ID, priority (P1=must-have, P2=should-have, P3=nice-to-have), and acceptance criteria. Do not skip any P1 requirement.

3.1 Authentication Module

ID	Requirement	Priority	Acceptance Criteria
FR-01	Youth and Job Providers can register with email + password OR phone number (OTP)	P1	User record created in Firebase Auth AND users/{uid} Firestore doc; role field set correctly
FR-02	Users can log in with email/password or phone OTP	P1	Firebase Auth token issued; user redirected to role-appropriate home screen
FR-03	Forgot Password — email reset link	P1	Firebase Auth sendPasswordResetEmail fires; user receives email
FR-04	Persistent login — user stays logged in across app restarts	P1	FirebaseAuth.instance.authStateChanges() persists session
FR-05	User can log out from profile screen	P1	Firebase signOut() called; all local state cleared; app navigates to login
FR-06	Role selection at registration (Youth / Job Provider)	P1	role field in Firestore users doc is 'youth' or 'provider'; routing differs

3.2 Youth Profile Module

ID	Requirement	Priority	Acceptance Criteria
FR-07	Youth can create and edit their profile: full name, photo, bio, skills (multi-select), location	P1	youth_profiles/{uid} doc created/updated in Firestore; photo uploaded to Firebase Storage
FR-08	Skills are selected from a predefined taxonomy (see Appendix A)	P1	Skill chips rendered from skills_taxonomy collection; user can multi-select up to 10
FR-09	Location is captured as a GeoPoint (lat/lng) via device GPS or manual map pin	P1	GeoPoint stored in youth_profiles.location; Google Maps picker used
FR-10	Profile displays accumulated rating average and jobs completed count	P1	Computed from ratings collection; shown as star rating widget
FR-11	Youth can set availability status: Available / Busy / Inactive	P2	status field in youth_profiles; affects matching engine inclusion

3.3 Job Provider Module

ID	Requirement	Priority	Acceptance Criteria
FR-12	Job Provider can create and edit their business/personal profile	P1	providers/{uid} doc in Firestore with name, photo, contact, location, rating
FR-13	Job Provider can post a new job listing	P1	jobs/{jobId} doc created with all required fields (see Section 6.2)
FR-14	Job post includes: title, description, skills_required[], location (GeoPoint), pay_amount, pay_type, deadline, category	P1	All fields validated before Firestore write; location requires map pin
FR-15	Job Provider can edit or close/delete their job listing	P1	jobs/{jobId} updated with status='closed'; soft delete only
FR-16	Job Provider can view list of applicants for each job	P1	Query applications where job_id==jobId; show youth profile summary
FR-17	Job Provider can accept or reject individual applications	P1	applications/{applId}.status updated to 'accepted' or 'rejected'; FCM notification sent to youth

3.4 Job Discovery & Matching Module

ID	Requirement	Priority	Acceptance Criteria
FR-18	Youth can browse all open job listings in a paginated card feed	P1	Firestore query jobs where status=='open', ordered by created_at desc, paginated 15/page
FR-19	Youth can search jobs by keyword (title/description full-text)	P1	Client-side Firestore query using >= and <= on title field OR Algolia integration (P2)
FR-20	Youth can filter jobs by: category, max distance, pay range	P2	Filter applied as Firestore compound query; distance filter computed client-side from GeoPoints
FR-21	Matching Engine — Cloud Function triggers on new job post	P1	See Section 7.1 for full algorithm specification
FR-22	Matched youth receive FCM push notification with job title and pay	P1	FCM message delivered within 10 seconds of job post creation
FR-23	Youth can view full job detail screen	P1	Job title, description, skills chips, location map snippet, provider info, pay, deadline shown

3.5 Application Module

ID	Requirement	Priority	Acceptance Criteria
FR-24	Youth can apply for a job with an optional cover message	P1	applications/{applId} doc created; status='pending'; job provider notified via FCM
FR-25	Youth cannot apply for the same job twice	P1	Check existing application before write; show 'Already Applied' state on button
FR-26	Youth can withdraw a pending	P2	applications/{applId}.status updated to

ID	Requirement	Priority	Acceptance Criteria
	application		'withdrawn'
FR-27	Youth can view all their applications with status badges	P1	Query applications where youth_id==uid; display Pending/Accepted/Rejected badges
FR-28	Job Provider receives in-app and push notification on new application	P1	FCM notification to provider's device token; notification stored in notifications/{uid}

3.6 Ratings & Reviews Module

ID	Requirement	Priority	Acceptance Criteria
FR-29	After job completion, Job Provider can rate youth (1–5 stars + comment)	P1	ratings/{ratingId} doc created; youth_profiles.rating_avg updated via Cloud Function
FR-30	After job completion, Youth can rate Job Provider (1–5 stars + comment)	P1	ratings/{ratingId} doc created; providers.rating_avg updated
FR-31	Ratings are displayed on both youth and provider profiles	P1	Star widget + numeric average + recent review list shown on profile screens
FR-32	A user can only rate once per job	P1	Firestore rule: check existing rating for this job_id + rater_id combination

3.7 Notifications Module

ID	Requirement	Priority	Acceptance Criteria
FR-33	All FCM notifications are stored in notifications/{uid}/{notifId} Firestore docs	P1	Each notification has: type, message, job_id (ref), is_read, created_at
FR-34	In-app Notifications screen shows all notifications with read/unread state	P1	Unread count badge on bottom nav; tap marks as read (is_read=true)
FR-35	Notification types: job_match, application_received, application_accepted, application_rejected, rating_received	P1	type field used to determine icon and navigation target on tap

3.8 Admin Module

ID	Requirement	Priority	Acceptance Criteria
FR-36	Admin can view list of all users with filter by role	P1	Admin Flutter web app queries users collection; role filter dropdown

ID	Requirement	Priority	Acceptance Criteria
FR-37	Admin can suspend or reactivate a user account	P1	users/{uid}.status='suspended'; Firestore rule denies suspended users write access
FR-38	Admin can view all job listings and remove inappropriate ones	P1	Admin query jobs collection; delete/close action available
FR-39	Admin can view platform analytics: total users, total jobs, total applications, match rate	P2	Aggregated from Firestore or Firebase Analytics dashboard

4. NON-FUNCTIONAL REQUIREMENTS (NFR)

AI AGENT: NFRs are enforced at architecture level, not just code comments. Firebase Security Rules enforce NFR-03. Firestore indexes enforce NFR-01. Flutter's flutter_localizations enforces NFR-09.

ID	Category	Requirement	Measure / Implementation
NFR-01	Performance	Job listing screen loads within 2 seconds on 3G	Firestore pagination (15 docs/page); Firestore composite indexes on jobs(status, created_at); Flutter ListView.builder for lazy rendering
NFR-02	Performance	Matching Cloud Function executes within 5 seconds	Firestore array-contains-any query (max 10 skills); Haversine distance computed in Node.js; FCM batch send
NFR-03	Security	All Firestore reads/writes governed by Security Rules	Firebase Security Rules enforce: (1) authenticated access only, (2) role-based access, (3) owner-only profile writes — see Section 13
NFR-04	Security	Passwords never stored in Firestore	Firebase Auth manages credentials; Firestore only stores uid-referenced profile data
NFR-05	Scalability	Support 10,000 concurrent users without degradation	Firebase auto-scales; Firestore horizontal sharding; Cloud Functions auto-scale on demand
NFR-06	Reliability	99.5% uptime	Firebase SLA 99.95%; offline read cache via Firestore persistence (enablePersistence: true)
NFR-07	Usability	ISO 9241-11 compliant — effective, efficient, satisfying	Minimum tap target 48×48dp; text contrast ≥4.5:1; max 3 taps to apply for job; user testing with 5+ youth
NFR-08	Portability	Single codebase for Android and iOS	Flutter; all platform-specific code in platform channels only if unavoidable
NFR-09	Localisation	Bilingual UI: Swahili (sw) + English (en)	flutter_localizations; ARB files for both locales; locale detection from device settings
NFR-10	Maintainability	Clean Architecture pattern	Feature-first folder structure; Repository pattern for Firestore; Provider/Riverpod for state
NFR-11	Accessibility	Support screen readers	Semantics() widgets on all interactive elements; meaningful content descriptions
NFR-12	Data Privacy	User location only used for matching — never shared	GeoPoint stored in Firestore; not exposed in public job listings; Security Rules block direct location reads by other users

5. SYSTEM ARCHITECTURE

5.1 Architecture Overview

Fursafy follows a four-layer architecture: Presentation (Flutter), Application Logic (Firebase Cloud Functions), Data (Firebase Services), and External/Infrastructure. All layers are decoupled through well-defined interfaces — Flutter talks to Firebase SDK, Cloud Functions talk to Firestore and FCM, external APIs (Google Maps, SMS) are called from Cloud Functions only.

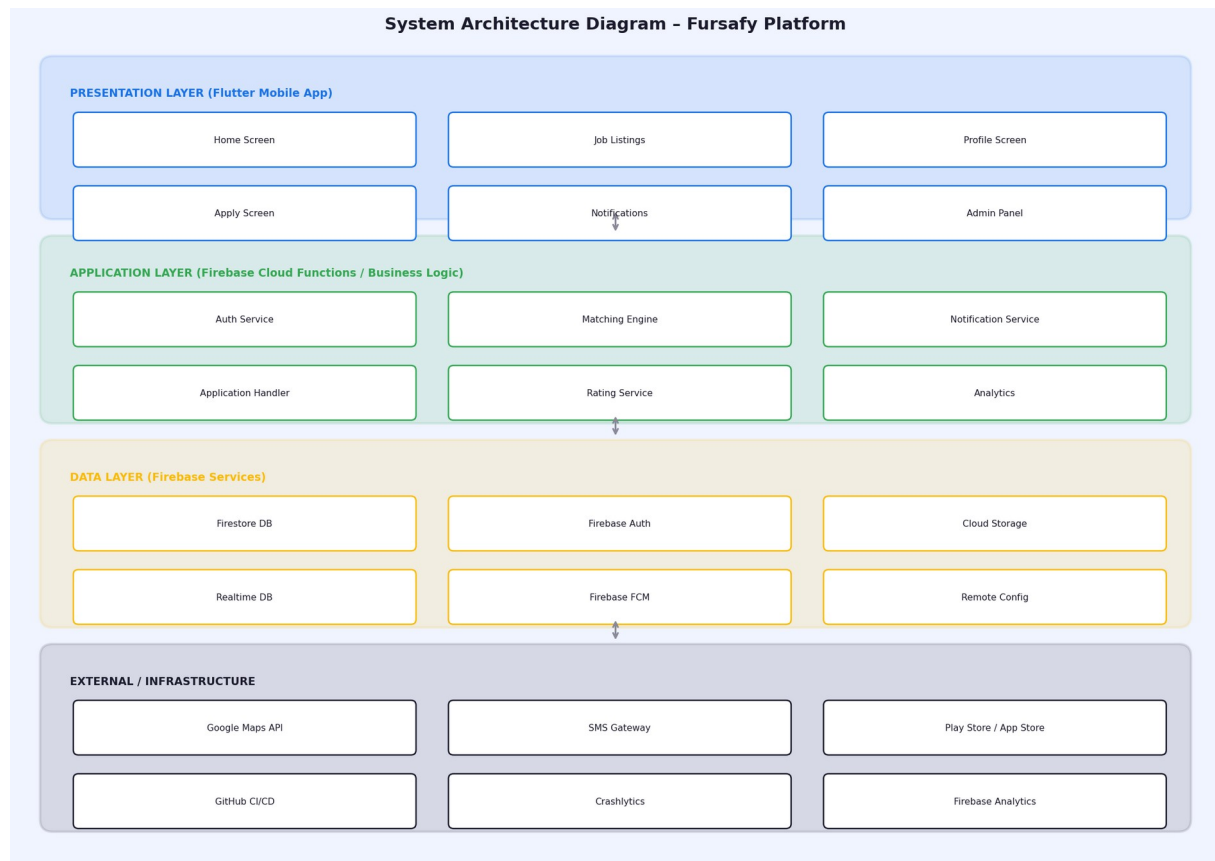


Figure 5.1 – Four-Layer System Architecture

5.2 Layer Specifications

Layer 1 — Presentation (Flutter App)

- Framework: Flutter 3.x, Dart 3.x
- State Management: BloC
- Navigation: GoRouter 12.x (declarative, deep-link ready)
- UI Components: Generated by Antigravity via Stitch MCP (see Section 9)
- Local Storage: Hive (user preferences, offline cache)
- Maps: google_maps_flutter package
- HTTP: Dio (for any REST calls; prefer Firebase SDK over REST)

Layer 2 — Application Logic (Cloud Functions)

- Runtime: Node.js 20 (Firebase Cloud Functions v2)

- Trigger: Firestore onCreate on jobs/{jobId}
- Functions: matchingEngine(), updateRatingAverage(), sendStatusNotification()
- Packages: firebase-admin, geofire-common (Haversine), firebase-functions

Layer 3 — Data (Firebase Services)

- Firestore: Primary database — all collections defined in Section 6
- Firebase Auth: Email/password + phone OTP; custom claims for role
- Cloud Storage: Profile images (users/{uid}/avatar.jpg), job images
- Firebase FCM: Push notification delivery to registered device tokens
- Remote Config: Feature flags (e.g., enable_premium_tier: false)
- Firebase Analytics: Custom events (job_posted, job_applied, match_triggered)

Layer 4 — External / Infrastructure

- Google Maps Platform: Places API (location search), Maps SDK (map display), Geocoding API
- SMS Gateway (optional fallback): Africa's Talking SMS API for OTP on low-end devices
- GitHub Actions: CI/CD pipeline — lint → test → build → deploy Cloud Functions
- Firebase App Distribution: Beta distribution to test users
- Crashlytics: Production crash monitoring

5.3 Data Flow — Job Posting to Youth Notification

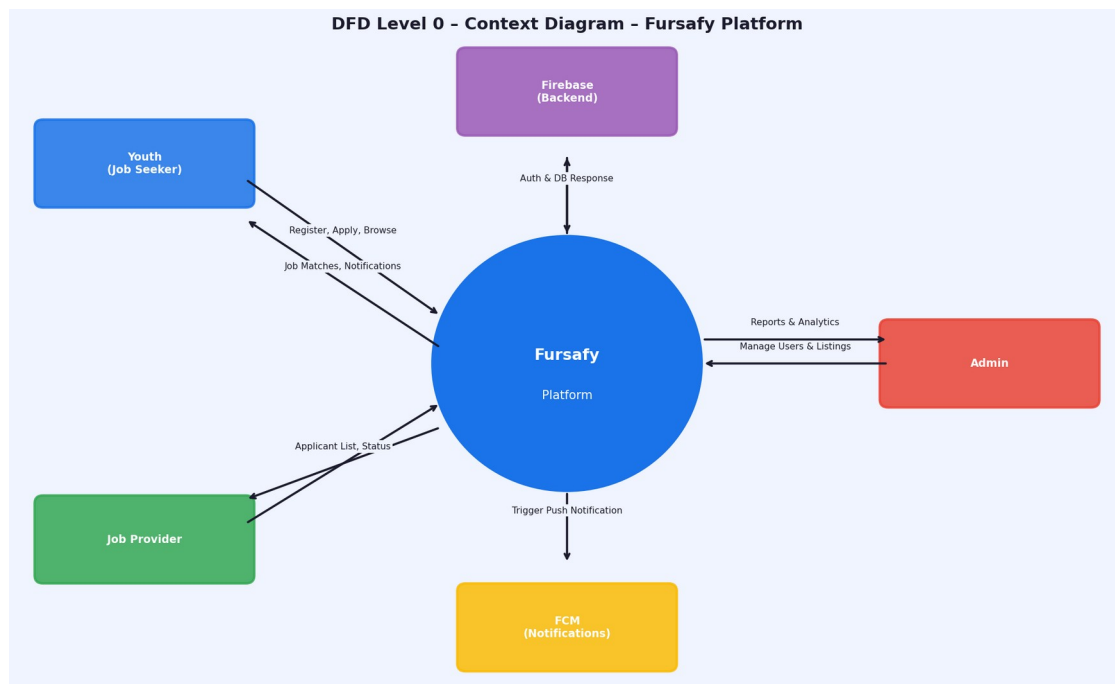


Figure 5.2 – DFD Level 0: System data flows

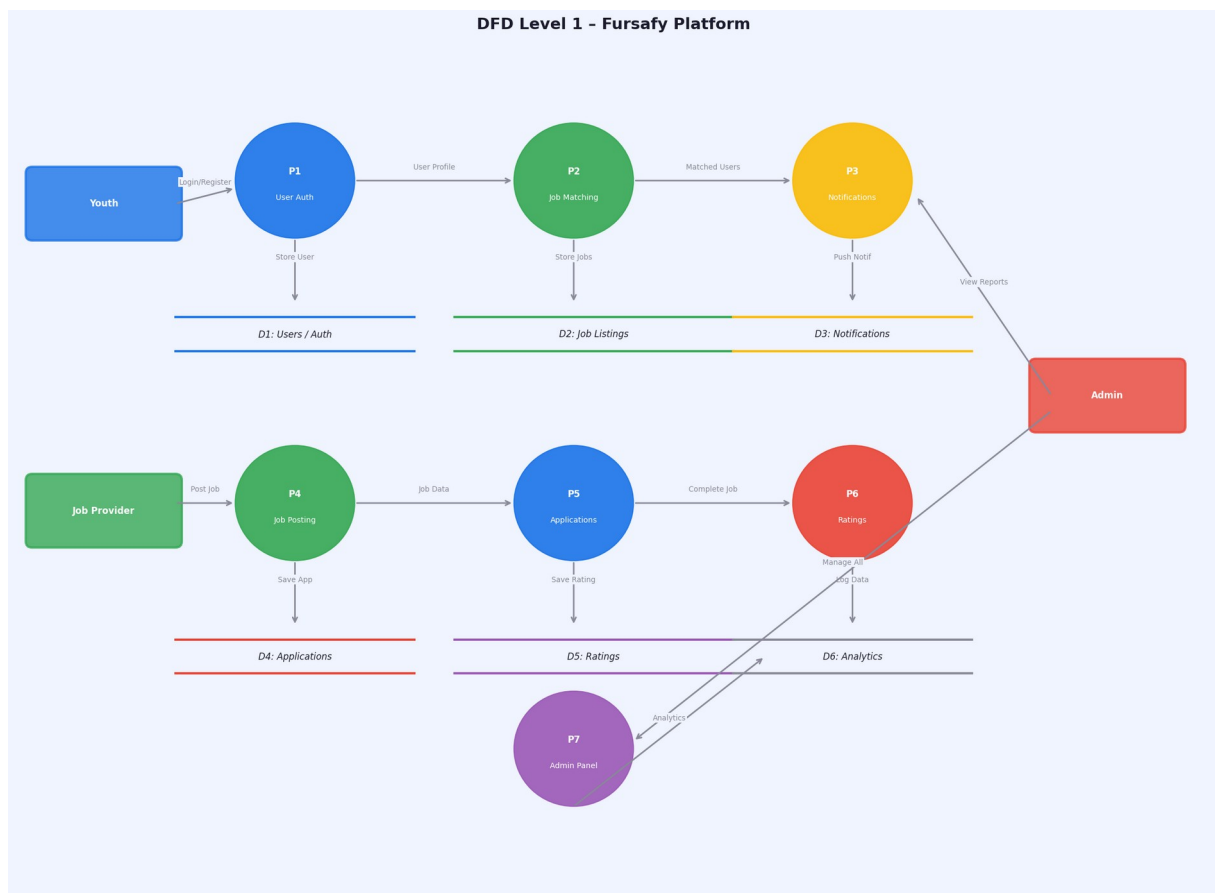


Figure 5.3 – DFD Level 1: Internal process decomposition

5.4 Sequence — Youth Applies for a Job

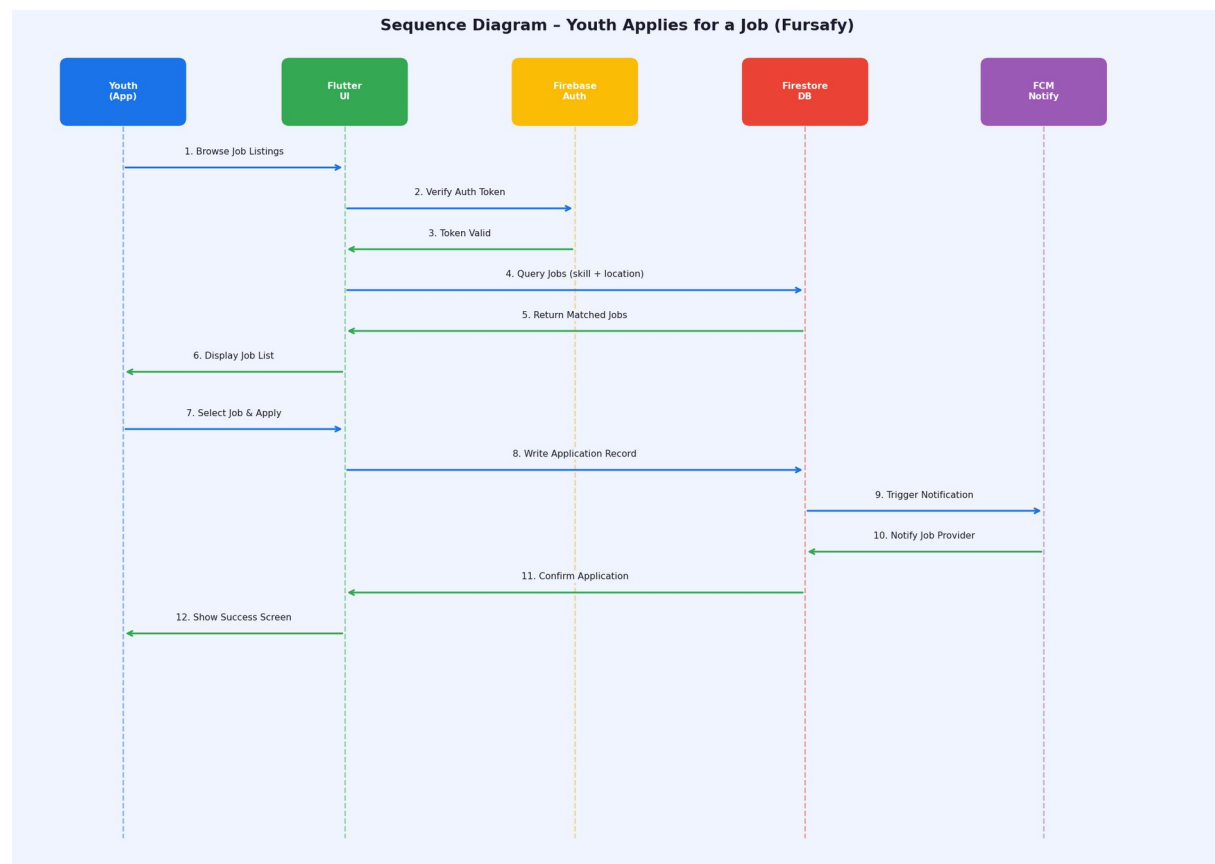


Figure 5.4 – Sequence diagram: apply flow (12 steps)

6. DATA MODELS & FIRESTORE SCHEMA

AI AGENT: Create ALL collections and documents in Firestore exactly as specified. Field names are camelCase. Use server timestamps (`FieldValue.serverTimestamp()`) for all `created_at` / `updated_at` fields. GeoPoints use Firebase `GeoPoint` type — never store `lat/ing` as separate strings.

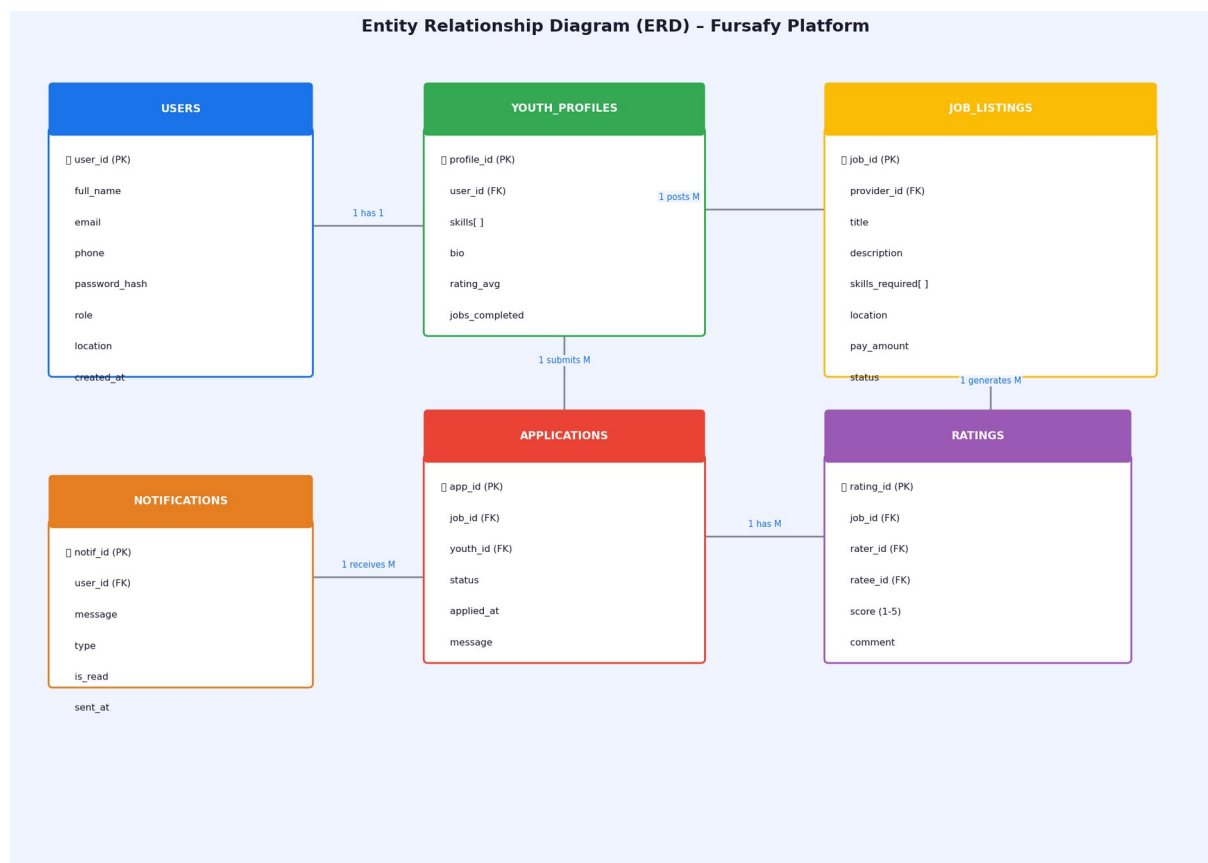


Figure 6.1 – Entity Relationship Diagram

6.1 Collection: users

Path: `users/{uid}`

Description: Base document created on registration for ALL user types

Field	Type	Required	Description
uid	String	Yes	Firestore Auth UID — matches document ID
fullName	String	Yes	Display name
email	String	Yes	Login email
phone	String	No	Phone number including country code (+255...)
role	Enum: youth provider admin	Yes	Determines routing and Security Rules

Field	Type	Required	Description
photoURL	String (Storage URL)	No	Profile photo — uploaded to Firebase Storage
location	GeoPoint	Yes	Captured at registration; updated on profile edit
status	Enum: active suspended inactive	Yes	Default: active
fcmToken	String	Yes	Updated on each app launch; used for push notifications
createdAt	Timestamp	Yes	FieldValue.serverTimestamp()
updatedAt	Timestamp	Yes	FieldValue.serverTimestamp()

6.2 Collection: jobs

Path: jobs/{jobId}

Description: Job listings posted by providers; triggers Matching Engine on create

Field	Type	Required	Description
jobId	String (auto)	Yes	Firestore auto-generated document ID
providerId	String (uid ref)	Yes	Job provider's Firebase Auth UID
title	String	Yes	Short job title (max 80 chars)
description	String	Yes	Full job description (max 1000 chars)
category	Enum	Yes	technical_repair cleaning tutoring construction delivery other
skillsRequired	Array<String>	Yes	Max 5 skills from taxonomy; used by Matching Engine
location	GeoPoint	Yes	Job site location; shown on map
locationLabel	String	Yes	Human-readable address (e.g., 'Arusha CBD, Sokoine Road')
payAmount	Number	Yes	Payment amount in Tanzanian Shillings (TSh)
payType	Enum: fixed hourly negotiable	Yes	Payment structure
deadline	Timestamp	Yes	Application deadline
status	Enum: open closed completed cancelled	Yes	Default: open
applicantCount	Number	Yes	Denormalised counter; incremented by Cloud Function
createdAt	Timestamp	Yes	FieldValue.serverTimestamp() — triggers Matching Engine
updatedAt	Timestamp	Yes	FieldValue.serverTimestamp()

6.3 Collection: youth_profiles

Path: youth_profiles/{uid}

Description: Extended profile for youth users; queried by Matching Engine

Field	Type	Required	Description
uid	String	Yes	Matches Firebase Auth UID and users/{uid}
skills	Array<String>	Yes	Selected from taxonomy; used in Matching Engine array-contains-any query
bio	String	No	Short biography (max 300 chars)
location	GeoPoint	Yes	Youth's current/home location for proximity matching
availabilityStatus	Enum: available busy inactive	Yes	Only available youth are matched
ratingAvg	Number	Yes	Computed average from ratings collection; default 0.0
jobsCompleted	Number	Yes	Counter incremented on job completion; default 0
portfolioURLs	Array<String>	No	Optional links to past work or certificates
updatedAt	Timestamp	Yes	FieldValue.serverTimestamp()

6.4 Collection: applications

Path: applications/{appld}

Field	Type	Required	Description
appld	String (auto)	Yes	Firestore auto-generated
jobId	String (ref)	Yes	Reference to jobs/{jobId}
youthId	String (uid ref)	Yes	Applicant's Firebase Auth UID
providerId	String (uid ref)	Yes	Job provider's UID (for Security Rules)
coverMessage	String	No	Optional message from youth (max 500 chars)
status	Enum: pending accepted rejected withdrawn	Yes	Default: pending
appliedAt	Timestamp	Yes	FieldValue.serverTimestamp()
updatedAt	Timestamp	Yes	FieldValue.serverTimestamp()

6.5 Collection: ratings

Path: ratings/{ratingId}

Field	Type	Required	Description
ratingId	String (auto)	Yes	Firestore auto-generated
jobId	String (ref)	Yes	The completed job this rating refers to
raterId	String (uid ref)	Yes	Who gave the rating
rateId	String (uid ref)	Yes	Who received the rating
raterRole	Enum: youth provider	Yes	Determines which profile to update
score	Number (1–5)	Yes	Integer star rating
comment	String	No	Written review (max 300 chars)
createdAt	Timestamp	Yes	FieldValue.serverTimestamp()

6.6 Collection: notifications

Path: notifications/{uid}/{notifId}

Note: Sub-collection under each user — enables efficient per-user queries

Field	Type	Required	Description
notifId	String (auto)	Yes	Firestore auto-generated
type	Enum	Yes	job_match application_received application_accepted application_rejected rating_received
message	String	Yes	Human-readable notification text
jobId	String (ref)	No	Related job (for deep-link navigation)
isRead	Boolean	Yes	Default: false; updated to true on user tap
createdAt	Timestamp	Yes	FieldValue.serverTimestamp()

6.7 Collection: skills_taxonomy

Path: skills_taxonomy/{skillId}

Description: Master list of available skills — seeded once at setup

Seed data (run once via Firebase Admin SDK or Firestore import):

```
{ id: 'plumbing',    label_en: 'Plumbing',    label_sw: 'Ufinyanzi wa Maji',    category: 'technical_repair' }
{ id: 'electrical',  label_en: 'Electrical Work', label_sw: 'Kazi ya Umeme',    category: 'technical_repair' }
{ id: 'carpentry',   label_en: 'Carpentry',    label_sw: 'Useremala', category: 'construction' }
{ id: 'cleaning',    label_en: 'Cleaning',     label_sw: 'Usafi',    category: 'cleaning' }
{ id: 'tutoring_math', label_en: 'Maths Tutoring', label_sw: 'Kufundisha Hisabati', category: 'tutoring' }
```

```
{ id: 'it_support',    label_en: 'IT Support',    label_sw: 'Msaada wa  
Teknolojia', category: 'technical_repair' }  
{ id: 'driving',      label_en: 'Driving',      label_sw: 'Udereva',  
category: 'delivery'  }  
{ id: 'painting',     label_en: 'Painting',     label_sw: 'Upigaji  
Rangi',    category: 'construction'  }  
{ id: 'cooking',      label_en: 'Cooking / Catering', 'label_sw': 'Kupika /  
Upishi',    category: 'other'          }  
{ id: 'garden',       label_en: 'Gardening',     label_sw: 'Bustani',  
category: 'cleaning'  }
```

BUSINESS LOGIC & ALGORITHMS

AI AGENT: Section 7.1 is the most critical business logic in the app. Implement EXACTLY as specified. Do not simplify or skip steps.

7.1 Matching Engine (Cloud Function)

Trigger: Firestore onCreate — fires when jobs/{jobId} is created

Runtime: Firebase Cloud Functions v2, Node.js 20

Function name: matchingEngine

Algorithm — Step by Step

6. Read the new job document: extract skillsRequired[] (array) and location (GeoPoint).
7. Query youth_profiles collection: WHERE skills array-contains-any skillsRequired[] AND availabilityStatus == 'available'. This returns all potentially matching youth.
8. For each candidate youth profile, compute Haversine distance between youth.location and job.location. Retain only candidates within MATCH_RADIUS_KM (default: 10km; configurable in Firebase Remote Config).
9. Score each remaining candidate: $\text{score} = (\text{matching_skill_count} \times 3) + (1 / \text{distance_km} \times 2) + (\text{ratingAvg} \times 1)$. Sort descending by score.
10. Cap matches at MAX_MATCHES (default: 50) to prevent FCM spam.
11. For each matched youth (in ranked order): (a) write a notifications/{youthId}/{notifId} document; (b) send FCM push notification to youth's fcmToken via Firebase Admin SDK; (c) log matching_triggered custom event in Firebase Analytics.

Cloud Function Code Scaffold

AI AGENT: Implement this exact function in functions/src/matchingEngine.ts

```
import * as functions from 'firebase-functions/v2';
import * as admin from 'firebase-admin';
const db = admin.firestore();
const MATCH_RADIUS_KM = 10;
const MAX_MATCHES = 50;

export const matchingEngine = functions.firestore.onDocumentCreated(
  'jobs/{jobId}', async (event) => {
    const job = event.data?.data();
    if (!job || job.status !== 'open') return;
    const { skillsRequired, location } = job;
    // Step 2: Query available youth with overlapping skills
    const snap = await db.collection('youth_profiles')
      .where('availabilityStatus', '==', 'available')
      .where('skills', 'array-contains-any', skillsRequired.slice(0,10))
      .get();
    // Step 3-4: Filter by distance, score, sort
    const candidates = snap.docs
      .map(d => ({ ...d.data(), uid: d.id }))
      .filter(y => haversineKm(location, y.location) <= MATCH_RADIUS_KM)
      .map(y => ({ ...y, score: calcScore(y, location, skillsRequired) }))
      .sort((a,b) => b.score - a.score)
      .slice(0, MAX_MATCHES);
```

```

// Step 5-6: Notify each matched youth
const batch = db.batch();
for (const youth of candidates) {
  const notifRef =
db.collection('notifications').doc(youth.uid).collection('notifs').doc();
  batch.set(notifRef, { type:'job_match', jobId: event.params.jobId,
    message: `New job match: ${job.title} - TSh ${job.payAmount}`,
    isRead: false, createdAt:
admin.firestore.FieldValue.serverTimestamp() });
}
await batch.commit();
// FCM batch send (from fcmTokens array)
const tokens = candidates.map(y => y.fcmToken).filter(Boolean);
if (tokens.length > 0) await admin.messaging().sendEachForMulticast({
  tokens, notification: { title: 'New Job Match!', body: job.title },
  data: { jobId: event.params.jobId, type: 'job_match' } });
}
);

```

7.2 Rating Average Update (Cloud Function)

Trigger: Firestore onCreate on ratings/{ratingId}

Logic: Re-query all ratings for rateId, compute average, update ratingAvg on youth_profiles or providers document

7.3 Application Status Notification (Cloud Function)

Trigger: Firestore onUpdate on applications/{applId}

Logic: If status changed to 'accepted' or 'rejected', send FCM notification to youth's fcmToken and write to notifications/{youthId}

7.4 Haversine Distance Formula

Used in Matching Engine (step 3). Implementation in TypeScript:

```

function haversineKm(p1: FirebaseFirestore.GeoPoint, p2:
FirebaseFirestore.GeoPoint): number {
  const R=6371, dLat=(p2.latitude-p1.latitude)*Math.PI/180;
  const dLon=(p2.longitude-p1.longitude)*Math.PI/180;
  const a=Math.sin(dLat/2)**2 + Math.cos(p1.latitude*Math.PI/180)
    *Math.cos(p2.latitude*Math.PI/180)*Math.sin(dLon/2)**2;
  return R*2*Math.atan2(Math.sqrt(a),Math.sqrt(1-a));
}

```

8. UI/UX SPECIFICATION & SCREEN INVENTORY

AI AGENT: All screens listed below MUST be generated by Antigravity via Stitch MCP (Section 9). Do not hand-code UI widgets. Use the HiFi mockup specifications below as the Antigravity prompt for each screen.

8.1 Visual Design Tokens

Token	Value	Usage
Primary Blue	#1A73E8	Buttons, links, active states, header accents
Accent Green	#34A853	Success states, Apply button, active badges
Warning Amber	#FBBC05	Pay amount badges, warnings
Danger Red	#EA4335	Rejection states, delete actions
Background Dark	#0A1628	Screen backgrounds (dark theme)
Surface Card	#122030	Card backgrounds
Border	#1E3A5F	Card borders, dividers
Text Primary	#FFFFFF	Main text on dark backgrounds
Text Secondary	#90CAF9	Subtitles, captions on dark backgrounds
Text Muted	#546E7A	Timestamps, placeholders
Font	Arial / Roboto	Body text — use system font stack
Heading Font	Arial Bold	All headings and CTA button text
Border Radius	12dp	All cards; 20dp for full-width buttons; 8dp for chips
Bottom Nav Height	64dp	Bottom navigation bar
Card Elevation	4dp	Job listing cards, profile cards

8.2 Screen Inventory

Complete list of all app screens, their route names, and Antigravity generation instructions:

#	Screen Name	Route	Auth Required	Role	Antigravity Prompt Key
S01	Splash / Loading	/splash	No	All	splash_screen
S02	Onboarding (3 slides)	/onboarding	No	All	onboarding_slides
S03	Login	/login	No	All	auth_login
S	Register — Step	/register/role	No	All	register_role

#	Screen Name	Route	Auth Required	Role	Antigravity Prompt Key
04	1 (role select)				
S05	Register — Step 2 (details)	/register/details	No	All	register_details
S06	Register — Step 3 (skills / location)	/register/profile	No	Youth	register_profile_youth
S07	Home / Job Feed	/home	Yes	Youth	home_job_feed
S08	Job Detail	/jobs/:jobId	Yes	Youth	job_detail
S09	Apply for Job (modal)	/jobs/:jobId/apply	Yes	Youth	apply_modal
S10	My Applications	/applications	Yes	Youth	my_applications
S11	Notifications	/notifications	Yes	All	notifications_list
S12	Youth Profile (own)	/profile	Yes	Youth	youth_profile_own
S13	Edit Profile / Skills	/profile/edit	Yes	Youth	profile_edit
S14	Provider Home / Dashboard	/provider/home	Yes	Provider	provider_dashboard
S15	Post New Job	/provider/jobs/new	Yes	Provider	post_job_form
S16	Edit Job	/provider/jobs/:jobId/edit	Yes	Provider	edit_job_form
S17	View Applicants	/provider/jobs/:jobId/applicants	Yes	Provider	applicants_list
S18	Youth Public Profile	/youth/:uid	Yes	Provider	youth_public_profile
S19	Rate Worker / Provider	/rate/:jobId	Yes	Both	rating_modal

#	Screen Name	Route	Auth Required	Role	Antigravity Prompt Key
9					
S20	Admin Dashboard	/admin	Yes	Admin	admin_dashboard
S21	Admin — User List	/admin/users	Yes	Admin	admin_user_list
S22	Admin — Job List	/admin/jobs	Yes	Admin	admin_job_list
S23	Search / Filter Jobs	/search	Yes	Youth	search_filter
S24	Map View (job locations)	/map	Yes	Youth	map_view

8.3 Key Screen Specifications

S07 — Home / Job Feed

- Top greeting: 'Hi [name]! 🙌 Discover Opportunities Near You'
- Search bar with filter icon — routes to S23
- Category filter chips: All | Tech | Cleaning | Construction | Tutoring | Other (horizontal scroll)
- Job cards (ListView.builder, paginated 15): emoji icon, job title (bold), location, time posted, pay badge (amber), Apply button (green)
- Bottom navigation: Home | Search | Post (FAB, provider only) | Notifications (badge) | Profile
- Empty state: illustration + 'No jobs near you yet. Check back soon!'

S08 — Job Detail

- Header: back arrow, 'Job Details' title
- Job title (large bold), location + pay + duration chips
- Provider card: avatar, name, star rating, jobs done count
- Description text (scrollable)
- Skills chips (from skillsRequired[])
- Primary CTA: 'Apply for This Job' (full-width green button) — disabled if already applied
- Secondary CTA: 'Save for Later' (outline button)

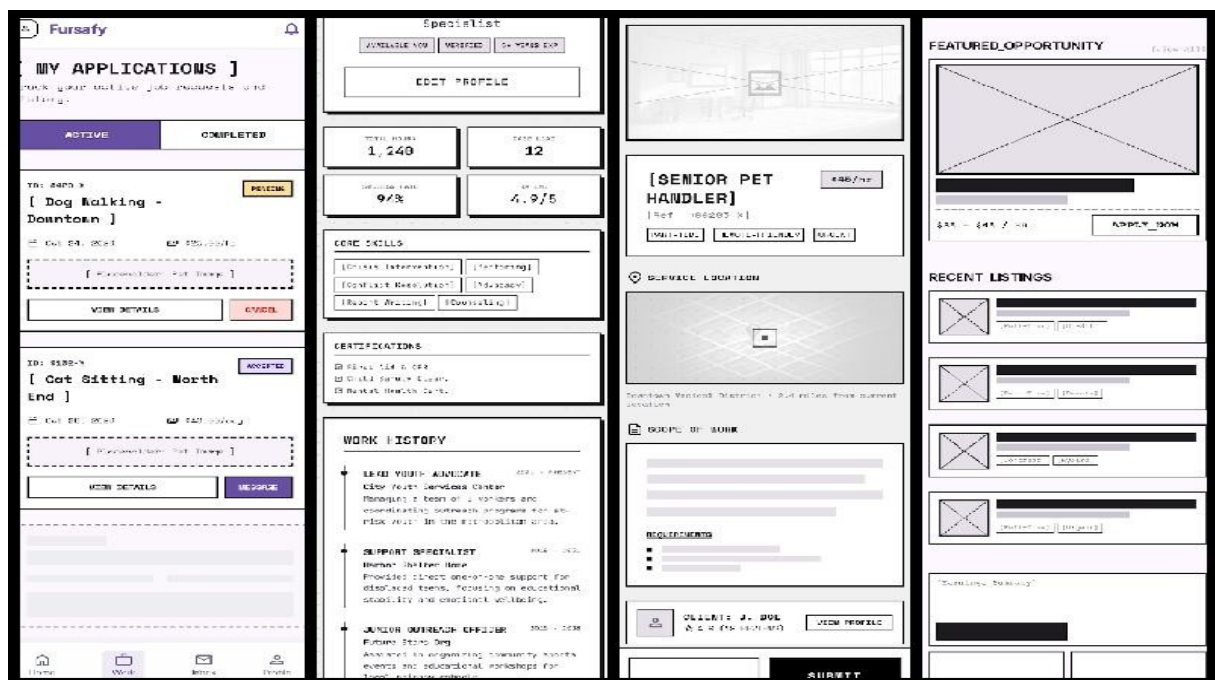


Figure 8.1 – High-Fidelity Mockups: Onboarding, Job Feed, Job Detail, Profile

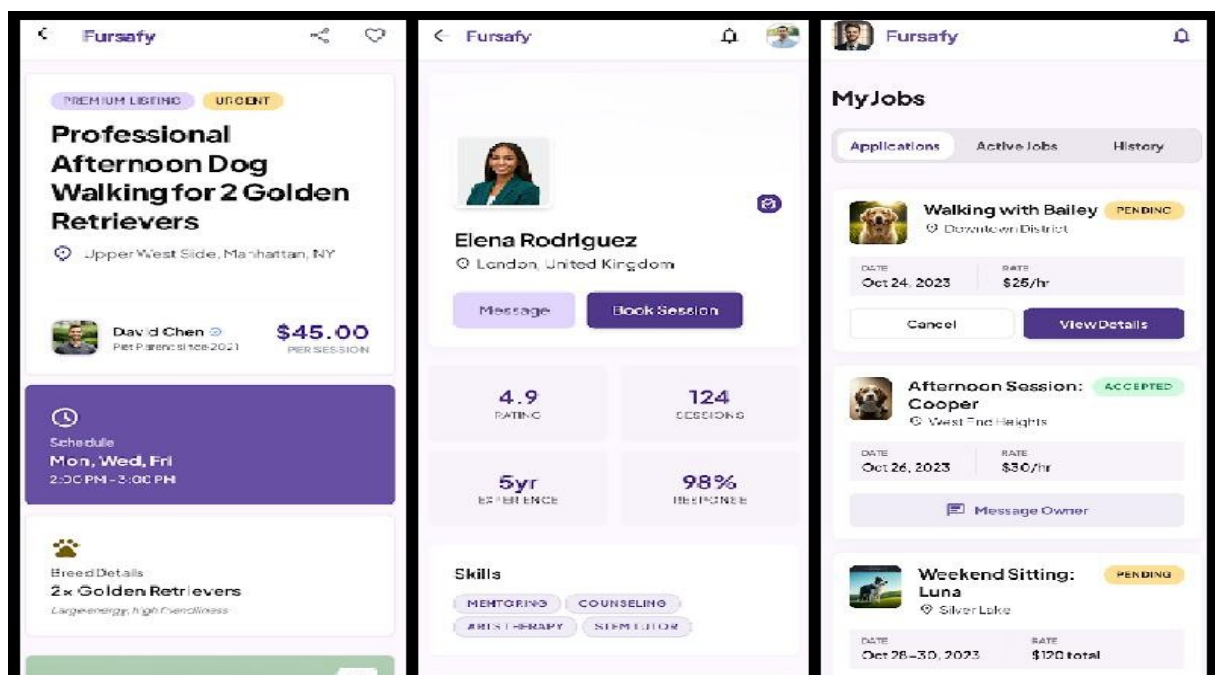


Figure 8.2 – Low-Fidelity Wireframes: structural layout reference

9. STITCH MCP + ANTIGRAVITY INTEGRATION GUIDE

AI AGENT: This section defines the EXACT workflow for generating all Fursafy UI screens using Antigravity via Stitch MCP. Follow these steps before writing any Flutter widget code. Antigravity generates production-ready Flutter widgets — use them directly.

9.1 What is Stitch MCP + Antigravity?

Stitch MCP is a Model Context Protocol server that connects AI coding agents to external services. Antigravity is a UI generation platform that takes design prompts and outputs production-ready Flutter widget code. Together, they allow an AI agent to generate all Fursafy screens without hand-coding any UI widgets.

9.2 Prerequisites

12. Stitch MCP server installed and running in the agent's environment.
13. Antigravity account provisioned and API key stored in environment variable `ANTIGRAVITY_API_KEY`.
14. Stitch MCP configured with the Antigravity connector — add to `mcp_servers` in agent config:

```
{ "name": "antigravity", "url": "https://stitch.antigravity.dev/mcp/v1" }
```
15. Design tokens from Section 8.1 uploaded to Antigravity as a shared design system named 'fursafy-design-system'.

9.3 Screen Generation Workflow

Step 1 — Upload Design System

Before generating any screen, post the design tokens (Section 8.1) to Antigravity as a design system. This ensures all generated screens use consistent colours, typography, and spacing.

```
// Via Stitch MCP tool call:
antigravity.createDesignSystem({
  name: 'fursafy-design-system',
  tokens: { primaryBlue:'#1A73E8', accentGreen:'#34A853', ... },
  fonts: ['Arial', 'Roboto'],
  theme: 'dark',
  borderRadius: { card:12, button:20, chip:8 }
})
```

Step 2 — Generate Each Screen

For each screen in the Screen Inventory (Section 8.2), call Antigravity via Stitch MCP using the screen's Antigravity Prompt Key and the specification from Section 8.3. Example for S07 (Home / Job Feed):

```
antigravity.generateScreen({
  designSystem: 'fursafy-design-system',
  screenKey: 'home_job_feed',
  framework: 'flutter',
  stateManagement: 'riverpod',
  navigation: 'go_router',
  specification: {
```

```
    layout: 'scrollable_feed',
    components: ['greeting_header', 'search_bar', 'category_chips',
'job_card_list', 'bottom_nav'],
    dataBindings: { jobList: 'StreamProvider<List<Job>>', userName: 'String' },
    actions: { onApplyTap: 'navigate_to_job_detail', onSearchTap:
'navigate_to_search' }
  }
})
```

Step 3 — Save Generated Widget Code

Antigravity returns a Flutter widget file. Save it to the correct path in the Flutter project:

```
lib/features/jobs/presentation/screens/home_screen.dart // S07
lib/features/jobs/presentation/screens/job_detail_screen.dart // S08
lib/features/auth/presentation/screens/login_screen.dart // S03
// ... one file per screen
```

Step 4 — Wire Business Logic

After UI generation, wire each screen to its Riverpod providers and Firestore repositories. Do not modify the generated UI code directly — use controllers and providers as the boundary layer.

10. FIREBASE CONFIGURATION & SETUP

AI AGENT: Follow these steps exactly before running the Flutter app. A misconfigured Firebase project will cause all features to fail.

10.1 Firebase Project Setup

16. Create a new Firebase project at console.firebase.google.com named 'fursafy-prod'.
17. Enable Blaze (pay-as-you-go) plan — required for Cloud Functions HTTP outbound calls.
18. Add Android app with package name: com.fursafy.app — download google-services.json to android/app/.
19. Add iOS app with bundle ID: com.fursafy.app — download GoogleService-Info.plist to ios/Runner/.

10.2 Services to Enable

Service	Console Location	Configuration
Firebase Auth	Authentication > Sign-in methods	Enable: Email/Password, Phone
Cloud Firestore	Firestore Database	Create in production mode; region: europe-west1 (closest to Tanzania)
Firebase Storage	Storage	Enable; set rules (see Section 13.3)
Cloud Functions	Functions	Enable; runtime Node.js 20
Firebase FCM	Project Settings > Cloud Messaging	Note the Server Key for Admin SDK
Google Analytics	Analytics	Enable; link to Firebase project
Crashlytics	Crashlytics	Enable after first app run
Remote Config	Remote Config	Create parameter: match_radius_km = 10

10.3 Required Flutter Packages

Add to pubspec.yaml:

```
dependencies:
  flutter: sdk: flutter
  firebase_core: ^3.x.x
  firebase_auth: ^5.x.x
  cloud_firestore: ^5.x.x
  firebase_storage: ^12.x.x
  firebase_messaging: ^15.x.x
  firebase_analytics: ^11.x.x
  firebase_crashlytics: ^4.x.x
  firebase_remote_config: ^5.x.x
  google_maps_flutter: ^2.x.x
  flutter_riverpod: ^2.x.x
  riverpod_annotation: ^2.x.x
  go_router: ^14.x.x
  hive_flutter: ^1.x.x
```

```
image_picker: ^1.x.x  
geolocator: ^13.x.x  
flutter_localizations: sdk: flutter  
intl: ^0.19.x
```

11. FLUTTER PROJECT STRUCTURE

AI AGENT: Use feature-first Clean Architecture. Each feature has its own data/domain/presentation layers. This structure is mandatory — do not use flat folder structure.

11.1 Folder Structure

```
lib/
├── main.dart                                # App entry point,
├── Firebase.initializeApp()
├── app/
│   ├── app.dart                            # MaterialApp.router with GoRouter
│   ├── router.dart                        # All GoRouter routes (24 screens)
│   └── theme.dart                          # AppTheme with design tokens
├── core/
│   ├── firebase/                          # FirebaseService, FCM handler
│   ├── error/                             # AppException, Failure classes
│   ├── utils/                             # Haversine util, validators, formatters
│   └── widgets/                           # Shared widgets: AppButton, AppCard,
│   └── SkillChip
├── features/
│   ├── auth/
│   │   ├── data/                          # AuthRepository, FirebaseAuthDataSource
│   │   ├── domain/                        # AuthRepository interface, User entity
│   │   └── presentation/                  # LoginScreen, RegisterScreen
│   └── (Antigravity generated)
│       ├── jobs/
│       │   ├── data/                      # JobRepository, FirestoreJobDataSource
│       │   ├── domain/                    # Job entity, Application entity
│       │   └── presentation/               # HomeScreen, JobDetailScreen
│       └── (Antigravity)
│           ├── profile/
│           │   ├── data/                  # ProfileRepository
│           │   ├── domain/                # YouthProfile, ProviderProfile entities
│           │   └── presentation/           # ProfileScreen, EditProfileScreen
│           ├── notifications/
│           │   ├── data/                  # NotificationRepository
│           │   └── presentation/           # NotificationsScreen
│           ├── ratings/
│           │   └── presentation/           # RatingModal
│           └── admin/
│               └── presentation/           # AdminDashboard, UserList, JobList
├── l10n/
│   ├── app_en.arb                         # English strings
│   └── app_sw.arb                         # Swahili strings
├── functions/                             # Firebase Cloud Functions
├── src/
│   ├── index.ts                           # Function exports
│   ├── matchingEngine.ts                  # FR-21 — full spec in Section 7.1
│   ├── updateRatingAverage.ts             # Section 7.2
│   └── sendStatusNotification.ts           # Section 7.3
└── package.json
```

API & INTEGRATION SPECIFICATIONS

12.1 Firebase Firestore Queries

All Firestore queries used in the app — agent must create corresponding composite indexes:

Screen / Feature	Collection	Query	Index Required
Home Job Feed	jobs	where status=='open' orderBy createdAt desc limit 15	status ASC, createdAt DESC
Search by category	jobs	where status=='open' where category==X orderBy createdAt desc	status ASC, category ASC, createdAt DESC
My Applications	applications	where youthId==uid orderBy appliedAt desc	youthId ASC, appliedAt DESC
Applicants for job	applications	where jobId==X orderBy appliedAt desc	jobId ASC, appliedAt DESC
Notifications	notifications/{uid}	orderBy createdAt desc limit 50	createdAt DESC
Ratings for user	ratings	where rateId==uid orderBy createdAt desc	rateId ASC, createdAt DESC
Matching Engine	youth_profiles	where status=='available' where skills array-contains-any []	Automatic for array queries

12.2 Google Maps Platform

API	Usage	Flutter Package
Maps SDK for Android/iOS	Display job location map on S08; show map pins for nearby jobs on S24	google_maps_flutter
Places API (Autocomplete)	Address search in job posting form (S15) and profile location setup	google_maps_flutter + http
Geocoding API	Convert address text to GeoPoint when provider types a location	Called from Cloud Function

12.3 FCM Notification Payload

All FCM messages must include a data payload for deep-link navigation:

```
{
  notification: { title: 'New Job Match!', body: 'Plumber Needed — TSh 25,000' },
  data: { type: 'job_match', jobId: 'abc123', screen: '/jobs/abc123' },
  android: { priority: 'high', notification: { sound: 'default', channelId: 'fursafy_jobs' } },
  apns: { payload: { aps: { sound: 'default', badge: 1 } } }
}
```

SECURITY & ACCESS CONTROL

AI AGENT: Deploy these Firestore Security Rules EXACTLY as written before running any app code. Rules are the primary security layer — a bug here exposes user data.

13.1 Firestore Security Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    // Helper functions
    function isSignedIn() { return request.auth != null; }
    function isOwner(uid) { return request.auth.uid == uid; }
    function getRole() { return get(/databases/{db}/documents/users/{request.auth.uid}).data.role; }
    function isYouth() { return getRole() == 'youth'; }
    function isProvider() { return getRole() == 'provider'; }
    function isAdmin() { return getRole() == 'admin'; }
    function isActive() { return get(/databases/{db}/documents/users/{request.auth.uid}).data.status == 'active'; }

    // users collection
    match /users/{uid} {
      allow read: if isSignedIn();
      allow write: if isOwner(uid) || isAdmin();
    }

    // jobs collection
    match /jobs/{jobId} {
      allow read: if isSignedIn();
      allow create: if isSignedIn() && isProvider() && isActive();
      allow update, delete: if isOwner(resource.data.providerId) || isAdmin();
    }

    // applications collection
    match /applications/{appId} {
      allow read: if isOwner(resource.data.youthId) || isOwner(resource.data.providerId) || isAdmin();
      allow create: if isSignedIn() && isYouth() && isActive();
      allow update: if isOwner(resource.data.youthId) || isOwner(resource.data.providerId);
    }

    // youth_profiles collection — location is private
    match /youth_profiles/{uid} {
      allow read: if isSignedIn();
      allow write: if isOwner(uid) || isAdmin();
    }

    // notifications — private to owner
    match /notifications/{uid}/{notifId} {
      allow read, write: if isOwner(uid);
    }

    // ratings
    match /ratings/{ratingId} {
      allow read: if isSignedIn();
      allow create: if isSignedIn() && isActive();
      allow update, delete: if false; // ratings are immutable
    }
  }
}
```

```
}
```

13.2 Firebase Auth Custom Claims

Set role claim on user creation via Cloud Function (onUserCreate trigger):

```
// Triggered when user doc is created - sets custom claim from role field
admin.auth().setCustomUserClaims(uid, { role: userData.role });
```

13.3 Firebase Storage Rules

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /users/{uid}/{allFiles=**} {
      allow read: if request.auth != null;
      allow write: if request.auth.uid == uid && request.resource.size < 5 *
1024 * 1024
                        && request.resource.contentType.matches('image/.*');
    }
  }
}
```

TESTING STRATEGY & TEST CASES

AI AGENT: Run ALL test cases before marking the build as complete. Unit tests run locally. Integration tests run against the Firebase Emulator Suite.

14.1 Testing Levels

Level	Scope	Tools	When to Run
Unit Tests	Individual Dart functions, Repository methods, Haversine formula	flutter test + mockito	On every code change (CI)
Widget Tests	Individual Flutter screens (rendered against mock providers)	flutter test + WidgetTester	On every UI change
Integration Tests	Full user flows on real/emulated device	flutter_test + Firebase Emulator	Before each sprint release
Cloud Function Tests	matchingEngine, updateRatingAverage, sendStatusNotification	jest + firebase-functions-test	On every Cloud Function change
Security Rules Tests	All Firestore Security Rules scenarios	Firebase Emulator + @firebase/rules-unit-testing	Before deploying rules
Usability Testing	5+ youth users using the app on real Android devices	Manual observation + SUS questionnaire	Before final submission

14.2 Critical Test Cases

TC ID	Module	Test Case	Expected Result	Pass Criteria
TC-01	Auth	Register as Youth with valid email	Account created in Firebase Auth + users/{uid} + youth_profiles/{uid}	All 3 docs exist; role='youth'
TC-02	Auth	Register as Provider with valid email	Account created + providers/{uid}	Doc exists; role='provider'
TC-03	Auth	Login with wrong password	Error shown: 'Wrong password'	No navigation; error message shown
TC-04	Matching	Post job with skills=['plumbing','electrical']; youth has skills=['plumbing']	Youth receives FCM notification within 10s	Notification in Firestore + FCM delivered
TC-05	Matching	Post job; youth is outside 10km radius	Youth does NOT receive notification	No notification doc created for out-of-range youth
TC-06	Matching	Post job; youth is busy	Youth not matched	No notification for busy youth

TC ID	Module	Test Case	Expected Result	Pass Criteria
6		status='busy'		
TC-07	Application	Youth applies for same job twice	Second apply blocked; button shows 'Already Applied'	No duplicate application doc
TC-08	Security Rules	Youth tries to write to another user's profile	Firestore permission denied	FirebaseException with PERMISSION_DENIED
TC-09	Security Rules	Unauthenticated user reads jobs	Firestore permission denied	FirebaseException with PERMISSION_DENIED
TC-10	Ratings	Provider rates youth after job completion	ratings doc created; youth_profiles.ratingAvg updated	ratingAvg = mean of all scores for that youth
TC-11	Notifications	Application accepted → youth notified	FCM sent + notifications/{youthId} doc created with type='application_accepted'	isRead=false; type correct
TC-12	Admin	Admin suspends user	users/{uid}.status='suspended'; user cannot post jobs	Firestore rule blocks write for suspended user

14.3 Usability Testing Protocol

Conduct usability testing with a minimum of 5 youth users from Arusha (age 18–30). Use the System Usability Scale (SUS) questionnaire after each session. Target SUS score ≥ 68 (above average). Test the following tasks:

20. Create a youth account and complete your profile with 3 skills.
21. Find a plumbing job near Arusha and apply for it.
22. Check your application status in My Applications.
23. Rate a job provider after a completed job.
24. Find and read a push notification, then navigate to the related job.

15. DEPLOYMENT & CI/CD

15.1 Environments

Environment	Firebase Project	Purpose
Development	fursafy-dev	Local development with Firebase Emulator Suite
Staging	fursafy-staging	Integration testing and supervisor review
Production	fursafy-prod	Final submission and live prototype

15.2 GitHub Actions CI/CD Pipeline

```
# .github/workflows/ci.yml
on: [push, pull_request]
jobs:
  test:
    steps:
      - flutter pub get
      - flutter analyze
      - flutter test
      - cd functions && npm test
  deploy_functions:
    if: branch == 'main'
    steps:
      - firebase deploy --only functions
  build_android:
    if: branch == 'main'
    steps:
      - flutter build appbundle --release
      - firebase appdistribution:distribute
build/app/outputs/bundle/release/app-release.aab
```

15.3 Pre-Submission Checklist

- All 24 screens generated by Antigravity and wired to business logic
- All 3 Cloud Functions deployed and passing tests
- Firestore Security Rules deployed and tested
- Firebase Storage Rules deployed
- Composite Firestore indexes created for all queries in Section 12.1
- skills_taxonomy collection seeded with all skill documents
- Bilingual strings complete for both en and sw ARB files
- SUS usability score ≥ 68 from 5+ test users
- All TC-01 through TC-12 test cases passing
- Crashlytics enabled and first session data visible

DEVELOPMENT CONSTRAINTS & ASSUMPTIONS

16.1 Technical Constraints

- Firestore free tier (Spark): 50,000 reads/day, 20,000 writes/day, 20,000 deletes/day. Monitor usage via Firebase console during testing.
- Cloud Functions require Blaze plan for outbound HTTP calls (FCM Admin SDK). Upgrade Firebase project before deploying functions.
- Google Maps API: Requires valid billing-enabled Google Cloud project. Restrict API key to app package name.
- Flutter 3.x required — do not downgrade. Check: flutter --version ≥ 3.19.
- Dart 3.x required for null safety and latest Riverpod features.

16.2 Academic Constraints

- This is an academic prototype — production scalability is designed for but not load-tested.
- Payment processing is OUT OF SCOPE for this submission per Section 1.6.
- The prototype will be evaluated by Eng. Abdul Kirobo at ATC — ensure the Admin dashboard is functional for demonstration.

16.3 Assumptions

- The AI agent has internet access and can install npm/pub packages from the allowed domains list.
- Stitch MCP + Antigravity connector is authenticated and functional before UI generation begins.
- Firebase project 'fursafy-prod' is already created and Blaze plan enabled before Cloud Functions deployment.
- Google Maps API key is provisioned and available as environment variable GOOGLE_MAPS_API_KEY.

17. GLOSSARY

Term	Definition
Antigravity	UI generation platform that produces production-ready Flutter widget code from design prompts
Cloud Function	Serverless function hosted on Firebase that runs on specific triggers (Firestore events, HTTP calls)
FCM	Firebase Cloud Messaging — Google's push notification service for Android and iOS
Firestore	Firebase's NoSQL cloud database, organised in collections and documents
GeoPoint	Firebase data type storing latitude and longitude as a single field
GoRouter	Flutter navigation package providing declarative routing with deep-link support
Haversine Distance	Mathematical formula for computing the shortest distance between two points on a sphere using lat/long
Matching Engine	Firebase Cloud Function that matches youth to job listings based on skills and proximity
MCP	Model Context Protocol — a standard allowing AI agents to connect to external tools and services
Riverpod	Flutter state management framework; AsyncNotifier and StreamProvider used in Fursafy
SRS	Software Requirements Specification — this document
Stitch MCP	MCP server providing AI agents access to Antigravity and other connected services
Skills Taxonomy	Predefined list of skill categories stored in skills_taxonomy Firestore collection
SUS	System Usability Scale — standardised questionnaire for measuring UI usability (score 0–100)
Waterfall	Sequential software development methodology: Requirements → Design → Implementation → Testing → Deployment
Youth	Primary user type: aged 18–35, registered as job seeker on Fursafy

REFERENCES

- Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2), 77–101.
- CloudDevs. (2024). Creating a job search app with Flutter: Filters and job listings. Retrieved from <https://clouddevs.com/flutter/job-search-app/>
- Creswell, J. W. (2014). *Research design: Qualitative, quantitative, and mixed methods approaches* (4th ed.). SAGE Publications.
- Daily News. (2025). Digital economy key to youth employment. Retrieved from <https://dailynews.co.tz>
- Daily News. (2026). Vijana platform: Empowering Tanzania's youth to shape future. Retrieved from <https://dailynews.co.tz>
- Evans, C., & Lean, J. (2025). Exploring university students' reasons for not working while studying: Implications for employability. *Journal of Education and Work*, 38(2).
- Google. (2023). Firebase documentation. Retrieved from <https://firebase.google.com/docs>
- Grunewald, A. (2022). A gig economy solution to boost employment in Africa. Brookings Institution. Retrieved from <https://www.brookings.edu>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.
- Ifakara Innovation Hub. (2021). Mchongoo: A digital marketplace bridging skills and job opportunities in Tanzania.
- International Labour Organization. (2022). Global employment trends for youth 2022. ILO.
- ISO 9241-11. (1998). Ergonomic requirements for office work with visual display terminals (VDTs) — Part 11: Guidance on usability. ISO.
- Mtebe, J., Kissaka, M. M., Raphael, C., & Stephen, J. K. (2020). Promoting youth employment through ICT in vocational education in Tanzania. *Journal of Learning for Development*, 7(1), 90–107.
- NACEWEB. (2024). Reimagining student employment. National Association of Colleges and Employers.
- Oates, B. J. (2006). *Researching information systems and computing*. SAGE Publications.
- Pressman, R. S. (2014). *Software engineering: A practitioner's approach* (8th ed.). McGraw-Hill.
- Saunders, M., Lewis, P., & Thornhill, A. (2019). *Research methods for business students* (8th ed.). Pearson.
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
- The Citizen. (2022). How Tanzanian university students are turning side hustles into survival strategies. Mwananchi Communications.
- The Citizen. (2023). DayWaka app connects Tanzanians to short-term jobs. Mwananchi Communications.
- Winn, H. L. (2021). The influence of part-time work on graduates' careers. *Higher Education, Skills and Work-based Learning*, 11(2), 372–387.
- World Bank. (2021). Tanzania economic update: Addressing youth unemployment through skills development. World Bank Group.
- World Bank. (2023). The promise and peril of online gig work in developing countries. World Bank Group.